Rake

Throughout this book, you've run your spec suite via the rspec command. But there's another common way to run your specs: using the Rake build tool. The rspec-core gem ships with an easy-to-configure Rake task. To enable it, add the following two lines to your project's Rakefile:

A1-rspec-and-wider-ecosystem/02/expense_tracker/Rakefile
```
require 'rspec/core/rake_task'
RSpec::Core::RakeTask.new(:spec)
```

This snippet defines a simple spec task that will run rspec with your configured defaults. You can then run it like so:

```
$ rake spec
```

Running RSpec this way adds some overhead. It takes time to load Rake itself, plus any libraries you need for other tasks in your Rakefile. A canned Rake task is also less flexible than the rspec command, since it does not let you customize individual runs through command-line options. That said, testing via Rake comes in handy in some situations:

- When you have specific sets of RSpec options you use together frequently
- When you want to run RSpec as one step in a multistep build process (such as on a continuous integration server)
- As a convenience to new developers on your project, who may expect running rake with no argument to build and test the code base completely

If you want to specify additional RSpec options, you can pass a block to RakeTask.new:

A1-rspec-and-wider-ecosystem/02/expense_tracker/Rakefile
```
require 'rspec/core/rake_task'

namespace :spec do
  desc 'Runs unit specs'
  RSpec::Core::RakeTask.new(:unit) do |t|
    t.pattern = 'spec/unit/**/*_spec.rb'
    t.rspec_opts = ['--profile']
  end
end
```

Here, we've defined a spec:unit task that runs all our unit specs with profiling enabled.